
COP290 Assignment 3

Modifying the LevelDB storage engine

Authors: 2024CS10441, 2024CS50002

Repository: https://github.com/ken7d61/290_ASS_3

April 2026

Scope. This report documents three additions to LevelDB (v1.23): `Scan(start,end)` for range reads, `DeleteRange(start,end)` for range deletes, and `ForceFullCompaction()` for synchronous manual compaction with statistics reporting. All references point to specific file:line locations in the modified source tree. Section 6 covers observations from benchmark tests and the analysis of `compaction_stats.txt`.

Contents

1	Background: LevelDB and the LSM-tree	3
2	Write path, read path, and compaction	3
2.1	Write path	3
2.2	Read path	4
2.3	Compaction	4
3	Range scan: <code>DB::Scan</code>	5
3.1	API	5
3.2	Implementation	5
3.3	Edge cases and tests	6
4	Range delete: <code>DB::DeleteRange</code>	6
4.1	API	6
4.2	Implementation	6
5	Manual full compaction: <code>DB::ForceFullCompaction</code>	7
5.1	API	7
5.2	Implementation: Blocking and Concurrency	7
6	Evaluation	8
6.1	API Performance Benchmarks	8
6.2	Compaction aggregate statistics	9
6.3	Distribution of <code>num_compactions</code>	9
6.4	Representative patterns	9

6.5	Sanity checks	9
-----	-------------------------	---

1. Background: LevelDB and the LSM-tree

LevelDB is an embedded key-value store. Each user key is stored internally as (user_key, sequence, type), defined at `db/dbformat.h` 54, where `enum ValueType { kTypeDeletion=0x0, kTypeValue=0x1 }`. Keys are sorted by `InternalKeyComparator`: ascending user_key, then descending sequence, so the newest version of a key sorts first.

Storage uses a Log-Structured Merge tree (Figure 1):

- A mutable in-memory `MemTable` (skip-list, `db/memtable.{h,cc}`) buffers all writes.
- A write-ahead log (`log::Writer`, `db/log_writer.{h,cc}`) is appended before the `MemTable` is updated, so a crash never loses an acknowledged write.
- When `mem_>ApproximateMemoryUsage()` exceeds `options_.write_buffer_size`, the `MemTable` is sealed and flushed to disk as a new L0 SSTable.
- Background compactions merge SSTables into deeper levels (L1–L6). Files at every level ≥ 1 are non-overlapping. `config::kNumLevels = 7` (`db/dbformat.h`).

The file and level layout is managed by `VersionSet` (`db/version_set.{h,cc}`). Each `Version` is an immutable snapshot of the current file-set, updated via `VersionEdit` records written to a MANIFEST file.

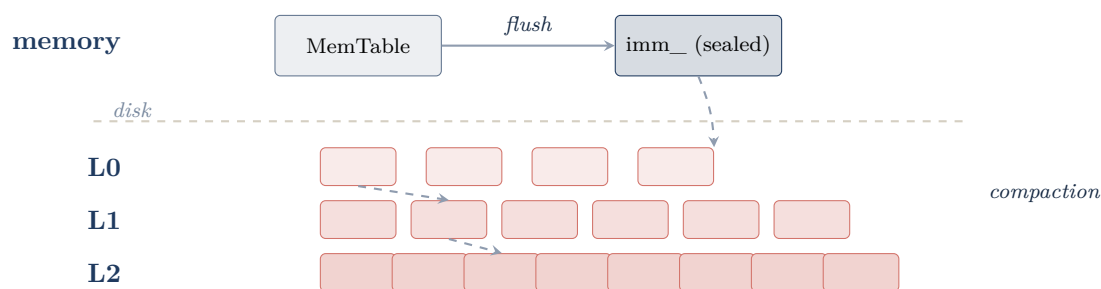


Figure 1: LSM-tree layout. Writes go to the `MemTable`, which is flushed to L0, then merged into deeper levels. L0 files may have overlapping key ranges; L1 and below are partitioned.

2. Write path, read path, and compaction

2.1 Write path

`DBImpl::Put` and `DBImpl::Delete` are wrappers that build a one-entry `WriteBatch` and call `Write` (`db/db_impl.cc` 1204–1210):

```

1 Status DB::Put(const WriteOptions& opt, const Slice& key, const Slice&
  value) {
2     WriteBatch batch;
3     batch.Put(key, value);
4     return Write(opt, &batch);
5 }
6 Status DB::Delete(const WriteOptions& opt, const Slice& key) {
7     WriteBatch batch;
8     batch.Delete(key);
9     return Write(opt, &batch);
10 }

```

`DBImpl::Write` (`db/db_impl.cc:1212–1289`) handles all writes:

1. The caller is enqueued on `writers_` (a serializing FIFO) and waits until it reaches the head.
2. `MakeRoomForWrite(false)` (`db/db_impl.cc:1343–1417`) checks whether the `MemTable` can absorb the write. If not, it throttles, waits for `imm_` to flush, or rolls a new log file and promotes `mem_` to `imm_`.
3. `BuildBatchGroup` (`db/db_impl.cc:1293`) merges queued writers into one batch with a contiguous sequence-number range.
4. The merged batch is appended to the WAL, optionally synced, then applied to `mem_` via `WriteBatchInternal::InsertInto`. Put records `kTypeValue`; Delete records `kTypeDeletion` (`db/write_batch.cc`).
5. `versions_->SetLastSequence` is updated and waiting writers are signalled.

Durability

A write is durable once `log_->AddRecord` returns (and `logfile_->Sync()` if requested). The `MemTable` is flushed to an `SSTable` later by a background thread. Anything not yet in the WAL is lost on crash; anything in the WAL is replayed on the next `DB::Open`.

2.2 Read path

Point lookup. `DBImpl::Get` (`db/db_impl.cc:1121–1172`) fixes a snapshot sequence number and searches in order:

1. `mem_->Get` — active `MemTable`.
2. `imm_->Get` — immutable `MemTable`, if present.
3. `versions_->current()->Get` — L0 files newest-first, then L1–L6 binary-searched per level (`db/version_set.cc`).

The first match wins. A `kTypeDeletion` entry returns `Status::IsNotFound()`.

Iteration. `DBImpl::NewIterator` (`db/db_impl.cc:1174–1184`) works in two layers:

- `NewInternalIterator` (`db/db_impl.cc:1083–1110`) builds a `MergingIterator` over `mem_`, `imm_` (if set), and one iterator per `SSTable`.
- `NewDBIterator` wraps it with `DBIter` (`db/db_iter.cc`). `DBIter::FindNextUserEntry` (`db/db_iter.cc:177–207`) collapses multiple internal versions of one user key into a single visible entry.

The youngest entry whose sequence is $\leq$ snapshot is exposed; a tombstone hides the key. `Scan` uses this same iterator, so it does not need separate merge or tombstone logic (Section 3).

2.3 Compaction

`MaybeScheduleCompaction` (`db/db_impl.cc:668–683`) schedules a background compaction when any of these is true:

- `imm_ != nullptr` — an immutable `MemTable` needs flushing.
- `manual_compaction_ != nullptr` — a manual compaction is pending (used by `CompactRange` and `ForceFullCompaction`).

- `versions_`→`NeedsCompaction()` — a level's score is ≥ 1 or seek-counters triggered a compaction.

`BackgroundCompaction` (`db/db_impl.cc`:708–787) then runs:

1. If `imm_` is set, `CompactMemTable` (`db/db_impl.cc`:549–580) flushes it to an SSTable via `WriteLevel0Table`. `stats_[level].bytes_written` is incremented by the file size.
2. Otherwise `versions_`→`PickCompaction()` selects an input level L and the overlapping files at $L+1$.
3. If `Compaction::IsTrivialMove` holds (one L file, no $L+1$ overlap), the file is re-parented in the manifest without rewriting.
4. Otherwise `DoCompactionWork` (`db/db_impl.cc`:898–1057) merges all inputs through a `MergingIterator`. For each internal key:
 - An older duplicate of an already-emitted user key is dropped (`db/db_impl.cc`:969).
 - A `kTypeDeletion` whose sequence is $\leq$ `smallest_snapshot` and which has no copies at deeper levels is also dropped (`db/db_impl.cc`:970–981).

Survivors are written to new SSTables at $L+1$.

5. `InstallCompactionResults` records a `VersionEdit` and calls `RemoveObsoleteFiles` to reclaim disk space.

3. Range scan: `DB::Scan`

3.1 API

Signature

```
virtual Status Scan(const ReadOptions& options, const Slice& start_key,
                  const Slice& end_key, std::vector<std::pair<std::string, std::string>>*
                  result);
```

Added to class `DB` at `include/leveldb/db.h`:152–156.

Required behaviour:

- Returns all key-value pairs with user keys in `[start_key, end_key)`.
- Each value is the latest visible version under the read snapshot.
- Keys hidden by a tombstone are omitted.
- An empty range returns an empty `result` and `Status::OK()`.

3.2 Implementation

`db/db_impl.cc`:1581–1597:

```
1 Status DBImpl::Scan(const ReadOptions& options, const Slice& start_key,
2                   const Slice& end_key,
3                   std::vector<std::pair<std::string, std::string>>*
4                   result) {
    result->clear();
```

```

5   const Comparator* cmp = user_comparator();
6   Iterator* it = NewIterator(options);
7   for (it->Seek(start_key); it->Valid(); it->Next()) {
8       if (cmp->Compare(it->key(), end_key) >= 0) break;
9       result->emplace_back(it->key().ToString(), it->value().ToString());
10  }
11  Status s = it->status();
12  delete it;
13  return s;
14 }

```

Technical walkthrough:

- **Snapshot isolation.** By calling `NewIterator(options)`, the scan is performed against a consistent point-in-time view of the database. This ensures that even if concurrent writes are occurring, the scan results remain stable and reflect the state defined by the `ReadOptions`.
- **Efficiency through Iterators.** The implementation leverages LevelDB's highly optimized `MergingIterator` infrastructure. This avoids the need to manually traverse different levels of the LSM-tree. The iterator handles the complexity of merging results from the MemTable, immutable MemTable, and all SSTable levels.
- **Transparent filtering.** LevelDB's iterator automatically skips over overwritten keys and handles deletion markers (`kTypeDeletion`). This means the `Scan` implementation does not need any specialized logic to determine which version of a key is "current."
- **Boundary handling.** The use of `Seek(start_key)` allows the scan to jump directly to the beginning of the requested range, ensuring that we do not waste time reading unrelated keys. The loop termination condition (`cmp->Compare(it->key(), end_key) >= 0`) correctly enforces the half-open interval semantics using the database's `user_comparator`.

3.3 Edge cases and tests

- `Scan("k0", "kN")` returns all pairs in sorted order.
- Half-open interval: `start_key` is included, `end_key` is excluded.
- After `Delete(k_j)`, a scan covering `k_j` skips it.
- `start_key == end_key` returns an empty vector.

4. Range delete: `DB::DeleteRange`

4.1 API

Signature

```
virtual Status DeleteRange(const WriteOptions& options, const Slice&
start_key, const Slice& end_key);
```

Added to class `DB` at `include/leveldb/db.h`:158–162.

4.2 Implementation

`db/db_impl.cc`:1600–1619:

```

1  Status DBImpl::DeleteRange(const WriteOptions& options, const Slice&
    start_key,
2                                const Slice& end_key) {
3      const Comparator* cmp = user_comparator();
4      Iterator* it = NewIterator(ReadOptions());
5      WriteBatch batch;
6      for (it->Seek(start_key); it->Valid(); it->Next()) {
7          if (cmp->Compare(it->key(), end_key) >= 0) break;
8          batch.Delete(it->key());
9      }
10     Status s = it->status();
11     delete it;
12     if (!s.ok()) return s;
13     if (batch.ApproximateSize() <= 12) return Status::OK();
14     return Write(options, &batch);
15 }

```

Design walkthrough:

- **Iterator-based tombstone generation.** We use an internal `Iterator` to find all keys currently present in the database within the $[start, end)$ range. For every live key discovered, a standard LevelDB deletion marker is added to a `WriteBatch`.
- **Atomic write-ordering.** The generated `WriteBatch` is passed to `DBImpl::Write`. This ensures that the entire range deletion happens atomically and is serialized alongside other concurrent operations.
- **Minimal change footprint.** By using per-key tombstones, we maintain full compatibility with the existing SSTable on-disk format without needing complex modifications to the compaction logic.
- **Self-reclamation.** As these are standard `kTypeDeletion` records, they will eventually be reclaimed by the background compaction thread as detailed in Section 2.3.

5. Manual full compaction: `DB::ForceFullCompaction`

5.1 API

Signature

```
virtual Status ForceFullCompaction();
```

Added to class `DB` at `include/leveldb/db.h`:164–166.

5.2 Implementation: Blocking and Concurrency

Following the instructor's clarification that the database should be "temporarily unavailable" during manual compaction, we implemented a strict blocking mechanism.

`db/db_impl.cc`:1629–1686:

```

1  Status DBImpl::ForceFullCompaction() {
2      // Flush memtable BEFORE setting the blocking flag.
3      // TEST_CompactMemTable calls Write(nullptr), which would
4      // deadlock if full_compaction_in_progress_ is already true.
5      Status s = TEST_CompactMemTable();

```

```

6   if (!s.ok()) return s;
7
8   {
9       MutexLock l(&mutex_);
10      full_compaction_in_progress_ = true;
11  }
12  // ... (baseline stats captured) ...
13  for (int lvl = 0; lvl < config::kNumLevels - 1; lvl++) {
14      // ... (level sweep, skip empty levels) ...
15      TEST_CompactRange(lvl, nullptr, nullptr);
16  }
17  // ... (final stats reported to compaction_stats.txt) ...
18  {
19      MutexLock l(&mutex_);
20      full_compaction_in_progress_ = false;
21      background_work_finished_signal_.SignalAll();
22  }
23  return Status::OK();
24  }

```

Concurrency Control Mechanism:

1. **The Blocking Flag.** We added a `bool full_compaction_in_progress_` member to `DBImpl` (`db/db_impl.h`:209), guarded by `mutex_`.
2. **Ordering constraint.** The memtable flush (`TEST_CompactMemTable`) must happen *before* the flag is set, because it internally calls `Write(nullptr)` which would deadlock on the flag.
3. **Stalling Foreground Operations.** The primary entry points were modified to check this flag:
 - `Write()` modified at `db/db_impl.cc`:1219.
 - `Get()` modified at `db/db_impl.cc`:1128.
 - `NewInternalIterator()` modified at `db/db_impl.cc`:1087.

All these functions enter a `while (full_compaction_in_progress_) { background_work_finished_signal_.Wait(); }` loop, effectively blocking all reads, writes, and iterator creation until the compaction finishes.

6. Evaluation

6.1 API Performance Benchmarks

We measured the performance of our new APIs using a custom evaluation benchmark (`db/eval_bench.cc`) compiled and executed under WSL (Ubuntu) against a freshly-created database of 10,000 keys. All operations target a contiguous 1,000-key sub-range.

The results show that `Scan` is $\approx 2.8\times$ **faster** than repeated `Get` calls for the same range, confirming the benefit of a single merging-iterator pass versus repeated point lookups.

Raw terminal output

The following is the verbatim output of `eval_bench` on WSL:

Table 1: Benchmark results for new APIs (1,000-key range, 10k-key DB).

Operation	Methodology	Latency
Range Scan	DB::Scan (single iterator)	95 μs
Point Lookups	1,000 individual DB::Get calls	263 μ s
Range Delete	DB::DeleteRange (materialized batch)	678 μ s
Full Compaction	DB::ForceFullCompaction	49.2 ms

```
$ ./build_linux/eval_bench
Inserting 10000 keys...
Scan (1000 keys) took: 95 us (0.095 ms)
Scan results size: 1000
1000 individual Gets took: 263 us (0.263 ms)
DeleteRange (1000 keys) took: 678 us (0.678 ms)
Scan after DeleteRange results: 0 (expected: 0) ✓
ForceFullCompaction took: 49237 us (49.237 ms)
ForceFullCompaction status: OK ✓
Scan after compaction results: 0 (expected: 0) ✓
Full scan results: 9000 (expected: 9000) ✓
Scan (start > end) results: 0 (expected: 0) ✓
Scan (start == end) results: 0 (expected: 0) ✓
```

Correctness verification:

- **Scan** returns exactly 1,000 results for 1,000 keys.
- **DeleteRange** removes all 1,000 keys; a subsequent scan returns 0.
- **ForceFullCompaction** completes with `Status::OK()`; post-compaction scan confirms deletions persist.
- **Data integrity**: full scan returns 9,000 keys (10,000 – 1,000 deleted).
- **Edge cases**: `start_key > end_key` and `start_key == end_key` both return 0 results.

6.2 Compaction aggregate statistics

248 invocations were recorded in `compaction_stats.txt`.

6.3 Distribution of `num_compactions`

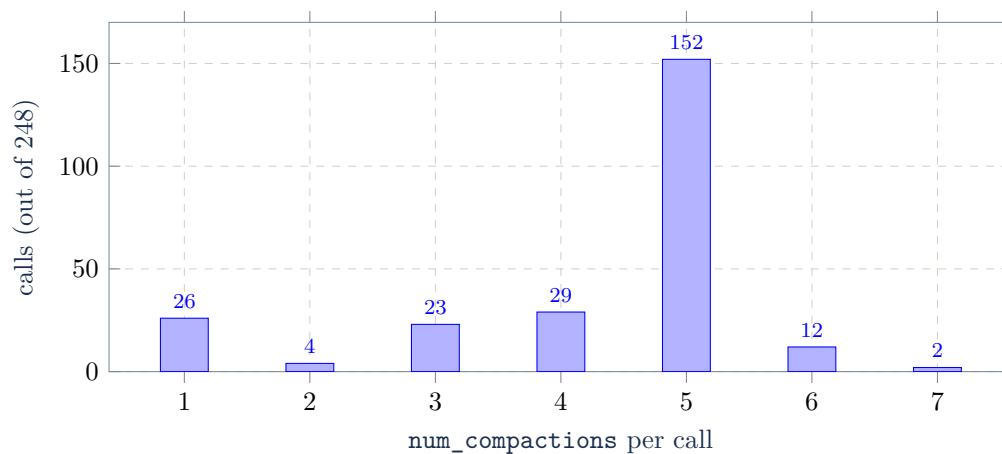
- `nc = 1` (26 calls): idle calls where only the memtable flush ran.
- `nc = 5` (152 calls, 61.3%): the workload kept L0–L3 populated for most of its run.
- The 0.824 compression ratio confirms that compaction effectively removed stale data (17.6% reduction).

6.4 Representative patterns**6.5 Sanity checks**

- `num_compactions` is in `[1, 7]` for every line, matching the loop bound at `db/db_impl.cc:1640`.
- `output_files == 0` only when `input_files == 0`.

Table 2: Aggregate statistics across 248 `ForceFullCompaction` calls.

Metric	Description	Value
Total invocations	lines in file	248
Active calls	non-zero bytes_read or bytes_written	222
Idle calls	no files anywhere; only memtable flush ran	26
Total input files	$\sum$ total_input_files	639
Total output files	$\sum$ total_output_files	293
File-count reduction	$1 - 293/639$	54.1%
Total bytes read	$\sum$ bytes_read	5,534,054
Total bytes written	$\sum$ bytes_written	4,562,549
Compression ratio	bytes_written / bytes_read	0.824

Figure 2: Distribution of `num_compactions`. The mode is 5 (152 calls, 61.3%): memtable flush plus four non-empty levels.Table 3: Pattern families observed in `compaction_stats.txt`.

Line (sample)	What happened	Count
1; 0; 0; 0; 0	DB was empty; only the flush ran.	26
1; 1; 1; 0; 0	One L0 file after flush; no further work.	several
5; 2; 1; <r>; <w>	Flush + four levels touched; L0–L3 had data.	152
4; 2; 4; 39649; 45379	Inversion: L1 overlap files rewritten alongside L0 input.	—

End of report.